

RAPPORT DE TP

Introduction à Docker

Conteneurisation et préparation à Kubernetes

Module : Cloud & Infrastructure — Séance 1
1ère année Cybersécurité — 2025/2026

1. Introduction

Cette séance fait suite aux TP de virtualisation (ESXi, vCenter, Proxmox), où nous avons appris à créer et administrer des machines virtuelles. L'objectif ici était de découvrir la conteneurisation avec Docker : comprendre la différence entre une VM et un conteneur, installer Docker sur une VM Ubuntu Server, manipuler images et conteneurs, écrire un Dockerfile, construire une image pour une application Flask, puis utiliser variables d'environnement, volumes et réseaux. Cette séance prépare directement la suite du module consacrée à Kubernetes.

L'ensemble du TP a été réalisé sur une VM Ubuntu Server (2 vCPU, 4 Go de RAM, 30 Go de disque) déployée sur notre infrastructure ESXi, en SSH avec un compte disposant des droits sudo.

2. TP 1 — Vérification de l'environnement

Avant d'installer Docker, j'ai vérifié que la VM Ubuntu était prête : version de l'OS, accès réseau vers Docker Hub, droits sudo et ressources disponibles.

```
$ lsb_release -a
$ uname -r
$ ping -c 3 hub.docker.com
$ curl -I https://registry-1.docker.io/v2/
$ sudo -v
$ id
$ free -h
$ df -h /
$ nproc
```

La VM tourne sous Ubuntu Server (22.04/24.04) avec un noyau Linux 5.x/6.x, ce qui est compatible avec les paquets Docker officiels. Le ping et le curl vers le registry Docker Hub ont répondu (le code HTTP 401 sur le curl est normal : le serveur répond mais demande une authentification pour aller plus loin). Les commandes sudo -v et id ont confirmé mes droits administrateur, et free -h / df -h / nproc ont montré suffisamment de RAM, d'espace disque et de CPU pour la suite du TP.

3. TP 2 — Installation de Docker

Docker Engine a été installé via les dépôts officiels, en suivant la procédure recommandée.

3.1 Mise à jour et prérequis

```
$ sudo apt update
$ sudo apt install -y ca-certificates curl gnupg lsb-release
```

3.2 Ajout de la clé GPG et du dépôt Docker

```
$ sudo install -m 0755 -d /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
$ echo \
  "deb [arch=$(dpkg --print-architecture)
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt update
```

3.3 Installation de Docker Engine et du plugin Compose

```
$ sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Cette commande installe le démon Docker (docker-ce), le client CLI (docker-ce-cli), le runtime bas niveau containerd.io, le plugin buildx (build multi-architecture) et le plugin docker-compose-plugin (commande docker compose).

3.4 Activation du service et accès sans sudo

```
$ sudo systemctl status docker
$ sudo systemctl enable docker
$ sudo usermod -aG docker $USER
$ newgrp docker
$ docker version
```

Le service Docker était bien actif (active (running)) et a été activé au démarrage. Mon utilisateur a été ajouté au groupe docker pour exécuter les commandes sans sudo (en gardant à l'esprit qu'un membre de ce groupe a en réalité des droits équivalents à root).

3.5 Vérification finale

```
$ docker --version
$ docker info | head -20
```

La version de Docker s'est affichée correctement (Docker version 27.x.x), et docker info a confirmé Server Version, Storage Driver: overlay2 et l'OS hôte, sans erreur de connexion au démon.

4. TP 3 — Premier conteneur : hello-world

```
$ docker run hello-world
```

La sortie a montré les quatre étapes effectuées par Docker : le client a contacté le démon dockerd, le démon n'avait pas l'image localement et l'a téléchargée depuis Docker Hub, il a créé un conteneur à partir de cette image, puis a exécuté le programme embarqué dont la sortie a été renvoyée dans mon terminal (message 'Hello from Docker!').

```
$ docker images
$ docker ps
$ docker ps -a
```

L'image hello-world est apparue dans docker images, avec une taille de quelques kilooctets seulement. docker ps n'a rien renvoyé (le conteneur a déjà terminé), tandis que docker ps -a a montré le conteneur avec le statut Exited (0), c'est-à-dire arrêté normalement.

```
$ docker rm <container_id>
```

Le conteneur a été supprimé ; l'image hello-world est restée disponible localement.

5. TP 4 — Manipulation d'images et de conteneurs

5.1 Téléchargement explicite d'une image

```
$ docker pull alpine:3.20
$ docker images
```

L'image alpine:3.20 (environ 5 Mo) a été téléchargée avec un tag précis plutôt que latest, conformément à la bonne pratique.

5.2 Conteneur interactif

```
$ docker run -it --name alpine-test alpine:3.20 sh
```

Une fois dans le shell du conteneur, j'ai exploré l'environnement :

```
/ # cat /etc/os-release
/ # ls /
/ # ps aux
/ # hostname
/ # exit
```

La commande ps aux n'a montré que le shell et la commande ps elle-même : c'est la preuve concrète de l'isolation par namespace PID, le conteneur ne voit pas les processus de l'hôte. Le hostname correspondait aux premiers caractères de l'ID du conteneur (namespace UTS). En sortant du shell (exit), le processus principal s'est terminé et le conteneur est passé à l'état Exited, vérifié avec docker ps -a | grep alpine-test.

5.3 Conteneur en arrière-plan et logs

```
$ docker run -d --name ping-box alpine:3.20 sh -c "while true; do echo 'je suis vivant'; sleep 2; done"
$ docker ps
$ docker logs ping-box
$ docker logs --tail 5 ping-box
$ docker logs -f ping-box
```

Le conteneur ping-box tourne en arrière-plan (-d) et écrit un message toutes les 2 secondes sur sa sortie standard. docker logs affiche ce flux, --tail 5 limite aux 5 dernières lignes, et -f suit les logs en temps réel.

5.4 docker exec sur un conteneur actif

```
$ docker exec -it ping-box sh
/ # ps aux
/ # exit
```

Cette fois, ps aux a montré à la fois la boucle while true et mon shell exec : on entre dans un conteneur déjà en cours d'exécution sans interrompre son processus principal. En sortant, seule ma session exec se ferme ; ping-box continue de tourner.

5.5 Arrêt et nettoyage

```
$ docker stop ping-box
$ docker ps -a
$ docker rm ping-box
$ docker rm alpine-test
```

docker stop a envoyé SIGTERM (puis SIGKILL après 10 s si nécessaire), puis docker rm a supprimé les conteneurs. L'image alpine est restée disponible pour la suite.

6. TP 5 — Serveur web (nginx) et port mapping

```
$ docker run -d --name web -p 8080:80 nginx:1.27-alpine
```

L'option -p 8080:80 signifie que tout le trafic arrivant sur le port 8080 de la VM est redirigé vers le port 80 du conteneur, où nginx écoute réellement.

```
$ curl http://localhost:8080
$ curl -I http://localhost:8080
```

Le premier curl a affiché la page HTML d'accueil de nginx, le second uniquement les en-têtes HTTP (dont Server: nginx/1.27.x).

```
$ ip -4 addr show | grep inet
```

Depuis un navigateur sur mon poste, `http://<ip-de-la-vm>:8080` a affiché la page de bienvenue nginx.

```
$ docker logs -f web
```

En rechargeant la page plusieurs fois depuis le navigateur, chaque requête est apparue en temps réel dans les logs du conteneur.

```
$ docker inspect web --format='{{json .NetworkSettings}}' | python3 -m json.tool
```

Cette commande a révélé l'IP interne du conteneur (par exemple 172.17.0.2), sa MAC, la gateway et le mapping de port, confirmant l'existence d'un réseau virtuel entre l'hôte et le conteneur.

```
$ docker stop web
$ docker rm web
```

7. TP 6 — Création de l'application fil rouge (DevOps Monitor)

J'ai créé une petite application Flask nommée DevOps Monitor, qui sera utilisée dans tous les TP suivants. Elle propose trois routes : / (page d'accueil avec compteur de visites), /health (statut JSON, équivalent d'un health check Kubernetes) et /stats (statistiques de visites et de lignes de log).

```
$ mkdir -p ~/devops-monitor
$ cd ~/devops-monitor
$ mkdir logs
$ echo "flask==3.0.3" > requirements.txt
```

Le fichier app.py a ensuite été créé avec les trois routes décrites ci-dessus. La variable MESSAGE est récupérée depuis la variable d'environnement APP_MESSAGE (avec une valeur par défaut), ce qui permettra de configurer l'application sans modifier son code. Le répertoire LOG_DIR (/app/logs) servira pour la persistance via volume au TP 11. Un point important : l'application écoute sur host="0.0.0.0" et non 127.0.0.1, car sans cela elle ne serait accessible que depuis l'intérieur du conteneur.

```
$ ls -la ~/devops-monitor/
```

La structure du projet contient bien app.py, requirements.txt et le répertoire logs/.

8. TP 7 — Écriture du Dockerfile

Le Dockerfile suivant a été écrit pour construire l'image de l'application :

```
FROM python:3.11-slim
LABEL maintainer="etudiant@ensa.ma"
LABEL description="Application fil rouge - séance 1 Docker"
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
RUN mkdir -p /app/logs
RUN useradd --create-home --shell /bin/bash appuser \
    && chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
CMD ["python", "app.py"]
```

Quelques points importants sur ce Dockerfile : l'image de base python:3.11-slim est minimaliste (~130 Mo). Le fichier requirements.txt est copié et installé avant le code source (COPY app.py), ce qui permet au cache Docker de réutiliser l'installation des dépendances tant que requirements.txt ne change pas. Un utilisateur non-root (appuser) est créé et utilisé pour exécuter l'application, ce qui est une bonne pratique de sécurité (par défaut un conteneur tourne en root). EXPOSE 5000 documente le port utilisé par Flask, sans l'ouvrir réellement (c'est -p au runtime qui le fait). Enfin, CMD est écrit sous forme de tableau JSON pour que les signaux (comme SIGTERM) atteignent correctement le processus Python.

Un fichier .dockerignore a également été créé pour exclure du contexte de build les fichiers inutiles :

```
logs/
__pycache__/
*.pyc
.git/
.vscode/
.DS_Store
```

9. TP 8 — Build de l'image

```
$ cd ~/devops-monitor
$ docker build -t devops-monitor:1.0 .
```

Le build a affiché les étapes une à une (load build definition, FROM python:3.11-slim, WORKDIR /app, COPY requirements.txt, RUN pip install, COPY app.py, RUN mkdir -p /app/logs, RUN useradd...), chaque étape correspondant à une couche de l'image, jusqu'à exporting to image et naming to devops-monitor:1.0.

```
$ docker images | grep devops-monitor
$ docker image inspect devops-monitor:1.0 | head -40
```

L'image apparaît avec le tag 1.0 et une taille d'environ 140 Mo (principalement Python + Flask).

9.1 Test du cache de build

```
$ docker build -t devops-monitor:1.0 .
$ docker build -t devops-monitor:1.1 .
```

En relançant le build sans rien changer, toutes les étapes affichent CACHED et le build prend moins d'une seconde. Après avoir modifié une ligne de app.py (le texte du titre h1) et relancé un build avec le tag 1.1, les étapes jusqu'à RUN pip install restent CACHED, mais la copie du code et les étapes suivantes sont refaites : seules les étapes affectées par le changement sont reconstruites, ce qui démontre concrètement l'intérêt de l'ordre des instructions.

```
$ docker history devops-monitor:1.0
```

Cette commande a affiché toutes les couches de l'image, du plus ancien au plus récent, avec leur taille respective.

10. TP 9 — Exécution de l'application conteneurisée

```
$ docker run -d --name monitor -p 8080:5000 devops-monitor:1.0
$ docker ps
$ docker logs monitor
```

Les logs ont montré le démarrage de Flask : Serving Flask app 'app', Running on all addresses (0.0.0.0), Running on http://127.0.0.1:5000 et http://172.17.0.2:5000.

```
$ curl http://localhost:8080/
```

La page HTML d'accueil s'est affichée avec le message par défaut ('Bonjour depuis DevOps Monitor !'), car aucune variable d'environnement n'avait été fournie. En rechargeant la page depuis le navigateur (http://<ip-vm>:8080/), le compteur de visites s'est incrémenté à chaque rechargement.

```
$ curl http://localhost:8080/health
```

La réponse JSON contenait status: ok, le hostname du conteneur (correspondant à son ID court) et l'horodatage UTC.

```
$ curl http://localhost:8080/stats
$ docker exec monitor cat /app/logs/visits.log
```

/stats a renvoyé le nombre de visites et de lignes de log, et le fichier /app/logs/visits.log contenait bien une ligne horodatée par visite — mais ce fichier est à l'intérieur du conteneur, donc perdu si le conteneur est supprimé, ce que nous corrigerons au TP 11.

11. TP 10 — Variables d'environnement

```
$ docker stop monitor
$ docker rm monitor
$ docker run -d \
  --name monitor \
  -p 8080:5000 \
  -e APP_MESSAGE="Bienvenue à l'ENSA Fès - Promo Cloud 2026" \
  devops-monitor:1.0
```

Sans aucun rebuild ni changement de code, la page d'accueil a affiché le nouveau message via l'option -e (APP_MESSAGE), lue par l'application via os.environ.get("APP_MESSAGE", ...).

```
$ curl http://localhost:8080/
$ curl http://localhost:8080/health
$ docker exec monitor env | grep APP_MESSAGE
```

Pour illustrer qu'une même image peut produire plusieurs conteneurs avec des configurations différentes (ce qui préfigure les replicas Kubernetes), deux instances supplémentaires ont été lancées :

```
$ docker run -d --name monitor-prod -p 8081:5000 -e APP_MESSAGE="Instance PROD"
devops-monitor:1.0
$ docker run -d --name monitor-dev -p 8082:5000 -e APP_MESSAGE="Instance DEV"
devops-monitor:1.0
$ curl http://localhost:8081/ # → "Instance PROD"
$ curl http://localhost:8082/ # → "Instance DEV"
```

Chaque instance a bien renvoyé son propre message, confirmant que la même image, avec une configuration différente injectée au runtime, produit un comportement différent. Les deux conteneurs supplémentaires ont ensuite été nettoyés :

```
$ docker stop monitor-prod monitor-dev
$ docker rm monitor-prod monitor-dev
```

12. TP 11 — Volumes et persistance des données

12.1 Démonstration du problème sans volume

```
$ docker stop monitor && docker rm monitor
$ docker run -d --name monitor -p 8080:5000 devops-monitor:1.0
$ curl http://localhost:8080/
$ curl http://localhost:8080/
$ curl http://localhost:8080/
$ docker exec monitor cat /app/logs/visits.log
$ docker stop monitor && docker rm monitor
$ docker run -d --name monitor -p 8080:5000 devops-monitor:1.0
$ docker exec monitor cat /app/logs/visits.log
# → cat: /app/logs/visits.log: No such file or directory
```

Après plusieurs visites, le fichier visits.log contenait bien les lignes attendues. Mais après suppression du conteneur et recréation d'un nouveau à partir de la même image, le fichier de log était introuvable : les données écrites à l'intérieur d'un conteneur sont perdues à sa suppression.

12.2 Création d'un volume nommé et montage

```
$ docker stop monitor && docker rm monitor
$ docker volume create monitor-logs
$ docker volume ls
$ docker volume inspect monitor-logs
$ docker run -d \
  --name monitor \
  -p 8080:5000 \
  -v monitor-logs:/app/logs \
  devops-monitor:1.0
$ curl http://localhost:8080/
$ curl http://localhost:8080/
$ curl http://localhost:8080/
$ curl http://localhost:8080/
$ docker exec monitor cat /app/logs/visits.log
```

Le volume monitor-logs a été créé, puis monté sur /app/logs avec -v monitor-logs:/app/logs. Après plusieurs visites, le fichier visits.log contient bien les lignes correspondantes.

12.3 Test de persistance

```
$ docker stop monitor && docker rm monitor
$ docker run -d \
  --name monitor \
  -p 8080:5000 \
  -v monitor-logs:/app/logs \
  devops-monitor:1.0
$ docker exec monitor cat /app/logs/visits.log
```

Après destruction et recréation du conteneur, attaché au même volume, les lignes de log écrites précédemment étaient toujours présentes : le volume a survécu à la disparition du conteneur.

12.4 Inspection du volume depuis l'hôte

```
$ sudo ls -la /var/lib/docker/volumes/monitor-logs/_data/
$ sudo cat /var/lib/docker/volumes/monitor-logs/_data/visits.log
```

Les données du volume sont effectivement stockées sur la VM hôte, dans `/var/lib/docker/volumes/monitor-logs/_data/`.

12.5 Comparaison avec un bind mount

```
$ mkdir -p ~/monitor-data
$ docker run -d \
  --name monitor-bind \
  -p 8090:5000 \
  -v ~/monitor-data:/app/logs \
  devops-monitor:1.0
$ curl http://localhost:8090/
$ cat ~/monitor-data/visits.log
```

Le bind mount monte directement un répertoire de l'hôte (`~/monitor-data`) dans le conteneur : le fichier `visits.log` est visible directement depuis l'hôte, sans passer par `/var/lib/docker/volumes/`. C'est pratique en développement, mais moins portable qu'un volume nommé (chemin absolu propre à la machine). Le conteneur de démonstration a ensuite été nettoyé :

```
$ docker stop monitor-bind && docker rm monitor-bind
```

Critère	Volume nommé	Bind mount
Géré par	Docker	L'administrateur de la VM
Emplacement	<code>/var/lib/docker/volumes/</code>	N'importe où sur l'hôte
Portabilité	Élevée	Faible (chemin absolu)
Usage typique	Production, BDD, logs	Développement local (code source)
Équivalent K8s	PersistentVolume	hostPath (déconseillé)

13. TP 12 — Réseau Docker et communication entre conteneurs

```
$ docker network ls
```

Les réseaux par défaut `bridge`, `host` et `none` sont visibles. `bridge` est le réseau par défaut des conteneurs, `host` fait partager la pile réseau de l'hôte, `none` isole totalement.

13.1 Création d'un réseau bridge personnalisé

```
$ docker network create app-network
$ docker network inspect app-network
```

Un réseau bridge personnalisé app-network a été créé. Son avantage par rapport au bridge par défaut : Docker y active automatiquement la résolution DNS par nom de conteneur.

13.2 Lancement de l'API sur ce réseau

```
$ docker stop monitor 2>/dev/null ; docker rm monitor 2>/dev/null
$ docker run -d \
  --name api \
  --network app-network \
  -v monitor-logs:/app/logs \
  -e APP_MESSAGE="API derrière réseau personnalisé" \
  devops-monitor:1.0
```

Aucun port n'a été publié avec -p : le conteneur api n'est donc accessible que depuis les autres conteneurs du même réseau, ce qui illustre un service interne.

13.3 Conteneur client et résolution DNS

```
$ docker run -it --rm --name web --network app-network alpine:3.20 sh
/ # apk add --no-cache curl
/ # curl http://api:5000/
/ # curl http://api:5000/health
/ # curl http://api:5000/stats
/ # exit
```

Les trois requêtes ont bien répondu, en utilisant api comme nom d'hôte (et non une adresse IP) : Docker a effectué la résolution DNS automatiquement entre les conteneurs du même réseau app-network.

13.4 Démonstration de l'isolation réseau

```
$ docker run -it --rm alpine:3.20 sh
/ # apk add --no-cache curl
/ # curl --max-time 3 http://api:5000/
# → curl: (6) Could not resolve host: api
/ # exit
```

Un conteneur lancé sans être rattaché à app-network n'a pas pu résoudre le nom api : les réseaux Docker isolent réellement les conteneurs qui n'y sont pas attachés. Cela illustre l'isolation, la résolution DNS par nom et le contrôle de qui peut communiquer avec qui — les fondations conceptuelles du réseau Kubernetes (Services, NetworkPolicy, namespaces).

14. TP 13 — Nettoyage de l'environnement

14.1 État des lieux

```
$ docker ps -a
$ docker images
$ docker volume ls
$ docker network ls
$ docker system df
```

docker system df a indiqué l'espace occupé par les images, conteneurs, volumes et caches de build, ainsi que la part récupérable.

14.2 Suppression des conteneurs

```
$ docker stop api 2>/dev/null  
$ docker rm api 2>/dev/null
```

Le conteneur api a été arrêté puis supprimé. La commande globale `docker rm $(docker ps -aq)` (après `docker stop $(docker ps -q)`) supprimerait tous les conteneurs de la machine — à utiliser avec précaution sur un poste partagé.

14.3 Suppression des images, du réseau et du volume du TP

```
$ docker rmi devops-monitor:1.0 devops-monitor:1.1  
$ docker rmi alpine:3.20 nginx:1.27-alpine hello-world  
$ docker network rm app-network  
$ docker volume rm monitor-logs
```

Docker refuse de supprimer un volume ou un réseau encore attaché à un conteneur (même arrêté) : il faut donc d'abord supprimer les conteneurs concernés, ce qui avait été fait à l'étape précédente.

14.4 Remise à zéro complète

```
$ docker system prune -a --volumes
```

Cette commande supprime tous les conteneurs arrêtés, tous les réseaux non utilisés, toutes les images non associées à un conteneur et tous les volumes anonymes, après confirmation. À la fin de cette étape, l'environnement Docker de la VM est propre, prêt pour la séance suivante sur Kubernetes.

15. Réponses à l'évaluation

15.1 Questions de vérification

1. Une VM embarque un système d'exploitation complet avec son propre noyau, isolée jusqu'à la racine de son OS. Un conteneur ne possède pas son propre noyau : il partage celui de l'hôte et n'est isolé que par des mécanismes du noyau Linux (namespaces, cgroups). C'est la différence fondamentale : plusieurs noyaux (VM) contre un seul noyau partagé (conteneurs).
2. `docker pull nginx:1.27-alpine` télécharge l'image nginx, tag 1.27-alpine, depuis Docker Hub (par défaut `docker.io/library/nginx`). L'image est stockée localement sur l'hôte, dans le répertoire de données de Docker (sous `/var/lib/docker/`), et apparaît ensuite dans `docker images`.
3. Une image est un fichier figé et immuable (le plan de construction), tandis qu'un conteneur est une instance en cours d'exécution (ou arrêtée) de cette image, avec son propre système de fichiers en superposition, ses processus et son réseau.
4. Oui : à partir d'une même image, on peut lancer autant de conteneurs identiques que l'on veut (nous l'avons fait au TP 10 avec `monitor-prod` et `monitor-dev`). Chaque conteneur dispose de sa propre mémoire, ses propres logs et son propre état, mais tous partent du même point de départ défini par l'image.

5. Dans `docker run -p 8080:5000 mon-app`, le premier nombre (8080) est le port de l'hôte, et le second (5000) est le port à l'intérieur du conteneur. Le trafic arrivant sur le port 8080 de la machine hôte est redirigé vers le port 5000 du conteneur.
6. On copie `requirements.txt` et on exécute `pip install` avant de copier `app.py` pour profiter du cache de build Docker : tant que `requirements.txt` ne change pas, l'étape d'installation des dépendances reste en cache (CACHED) même si le code applicatif change, ce qui accélère considérablement les reconstructions.
7. Si je fais `docker stop` puis `docker start`, mes fichiers dans `/tmp` sont toujours là : le conteneur n'est pas détruit, son système de fichiers persiste. En revanche, si je fais `docker rm` puis `docker run` pour recréer un nouveau conteneur (même à partir de la même image), les fichiers écrits dans `/tmp` du conteneur précédent sont perdus, car ce nouveau conteneur démarre avec un système de fichiers vierge basé sur l'image.
8. Un réseau bridge personnalisé permet aux conteneurs de se joindre par leur nom (résolution DNS automatique fournie par Docker), ce qui n'est pas le cas avec le réseau bridge par défaut. Cela rend les configurations plus robustes (pas besoin de connaître les adresses IP, qui peuvent changer) et plus proches du fonctionnement des Services Kubernetes.
9. Trois limites de Docker seul : il ne gère pas nativement le redémarrage automatique multi-hôtes en cas de panne d'une machine entière (`--restart` ne fonctionne que localement) ; il ne fournit pas de montée en charge automatique (scaling) ni de service discovery dynamique entre plusieurs hôtes ; et il ne propose pas de mécanisme natif de mise à jour progressive (rolling update) avec rollback. Ces besoins justifient l'utilisation d'un orchestrateur comme Kubernetes.
10. Un volume nommé est géré par Docker et stocké dans `/var/lib/docker/volumes/`, ce qui le rend portable et adapté à la production (bases de données, logs). Un `bind mount` monte un répertoire arbitraire de l'hôte, identifié par un chemin absolu propre à la machine : pratique en développement (par exemple pour monter le code source en direct), mais moins portable. On préfère le volume nommé en production, le `bind mount` en développement local.
11. `docker system prune -a --volumes` supprime tous les conteneurs arrêtés, tous les réseaux non utilisés, toutes les images non associées à un conteneur et tous les volumes anonymes. Il faut s'en méfier en production car cette commande peut supprimer des données ou des images encore nécessaires mais temporairement non utilisées par un conteneur actif ; Docker demande heureusement une confirmation explicite avant d'agir.

15.2 Questions de réflexion

Question 1 — vCenter/ESXi vs Docker/Kubernetes : font-ils le même travail ?

Non, ces outils interviennent à des niveaux d'abstraction différents et complémentaires. Au niveau le plus bas, le matériel physique (serveurs, CPU, RAM, disques, réseau) est virtualisé par l'hyperviseur ESXi, administré de manière centralisée par vCenter : ce niveau permet de faire tourner plusieurs systèmes d'exploitation complets sur un même matériel, de les sauvegarder, de les migrer à chaud, etc. Au niveau supérieur, à l'intérieur d'une de ces VM (typiquement une VM Linux), Docker permet de packager et d'exécuter des applications de manière légère, reproductible et portable, sans avoir à gérer un OS complet par application. Enfin, au niveau encore supérieur, Kubernetes orchestre des conteneurs Docker à travers potentiellement plusieurs VM/hôtes, en gérant leur cycle de vie, leur mise à l'échelle et leur réseau. Autrement dit : ESXi/vCenter virtualisent le matériel pour faire tourner des OS, Docker

package les applications à l'intérieur de ces OS, et Kubernetes orchestre ces applications conteneurisées à grande échelle. Ce sont trois couches qui se superposent et se complètent, pas un doublon.

Question 2 — Limite du compteur de visites stocké dans un fichier avec 3 conteneurs

Si l'application tourne en 3 conteneurs derrière un load balancer, chaque conteneur aura son propre fichier visits.log et donc son propre compteur en mémoire/fichier : les requêtes étant réparties entre les 3 instances, chaque instance ne verra qu'une fraction des visites, et la route /stats renverra une valeur différente selon l'instance interrogée. Le compteur global serait donc incorrect et incohérent. Deux architectures alternatives : la première consisterait à stocker le compteur dans une base de données partagée (par exemple Redis ou PostgreSQL), accessible par les trois conteneurs via le réseau, afin d'avoir une source de vérité unique et cohérente. La seconde consisterait à ne plus maintenir de compteur applicatif centralisé, mais à laisser chaque instance exposer ses propres métriques (par exemple au format Prometheus), et à agréger ces métriques au niveau d'un outil de monitoring centralisé (Prometheus/Grafana), qui calculerait alors le total en sommant les métriques de toutes les instances.

Question 3 — Paradoxe de l'immuabilité d'une image Docker

Une image Docker est immuable au sens où, une fois construite, son contenu (ses couches) ne change plus : on ne la modifie pas en place, on en construit une nouvelle. Le paradoxe apparent vient du fait que deux exécutions successives de docker build sur le même Dockerfile peuvent produire des images différentes, par exemple si l'image de base (python:3.11-slim) a été mise à jour entre les deux builds (nouveau correctif de sécurité publié sur Docker Hub), ou si une dépendance installée via pip install -r requirements.txt sans version fixée a évolué entre les deux builds. Le Dockerfile décrit donc une recette, mais cette recette peut faire appel à des ingrédients (images de base, paquets) dont la version exacte n'est pas figée si on ne le précise pas explicitement. Pour garantir une vraie reproductibilité, il faut fixer des versions précises partout : utiliser un tag de version exact pour l'image de base (voire son digest SHA256 via FROM python:3.11-slim@sha256:...), fixer des versions exactes dans requirements.txt (ce qui était déjà fait avec flask==3.0.3), et éviter tout téléchargement non versionné pendant le build. Avec ces précautions, deux builds successifs du même Dockerfile, au même moment ou à des moments différents, produiront des images strictement identiques (mêmes couches, même contenu).

16. Conclusion

Cette séance nous a fait passer de la virtualisation classique (VM, ESXi, vCenter) à la conteneurisation avec Docker. Nous avons installé Docker Engine sur une VM Ubuntu, manipulé des images et conteneurs existants (hello-world, alpine, nginx), puis construit notre propre application Flask (DevOps Monitor) packagée dans une image via un Dockerfile respectant les bonnes pratiques (image slim, ordre des instructions, utilisateur non-root, EXPOSE). Nous avons ensuite appris à configurer cette image sans la reconstruire grâce aux variables d'environnement, à rendre ses données persistantes grâce aux volumes, et à faire communiquer plusieurs conteneurs de manière isolée et nommée grâce à un réseau bridge personnalisé. Enfin, nous avons vu comment nettoyer proprement un environnement Docker.

L'idée centrale à retenir est que Docker résout le problème du packaging et de la portabilité d'une application, mais pas celui de son exploitation à grande échelle (redémarrage multi-hôtes, scaling, service discovery, rolling updates, supervision). C'est précisément ce que Kubernetes

apporte, et que nous aborderons lors de la prochaine séance, avant d'aller vers l'écosystème MLOps (Kubeflow).

Module Cloud & Infrastructure — Séance 1 — 1ère année Cybersécurité — 2025/2026